

[Universitatea / Facultatea — de completat]

[Specializarea / Anul — de completat]



DOCUMENTAȚIE TEHNICĂ

Aplicație de șah multiplayer client-server

Client JavaFX · Server Spring Boot · REST · MariaDB · Docker

Autor: [Nume și prenume student — de completat]

Grupa: [de completat]

Coordonator: [de completat]

Brașov, mai 2026

Cuprins

1. Introducere

1.1. Tema proiectului

Lucrarea de față prezintă proiectarea și implementarea unei aplicații de șah multiplayer de tip client-server. Aplicația permite ca doi jucători aflați pe instanțe separate de client să joace o partidă completă de șah, mutările fiind transmise și sincronizate printr-un server central. Pe lângă mecanica de bază a jocului, aplicația oferă gestionarea timpului pentru fiecare jucător, salvarea partidelor într-o bază de date, un istoric de mutări cu posibilitatea de navigare prin pozițiile anterioare, precum și o serie de finaluri speciale de joc, cum ar fi remiza, cedarea și semnalarea mutărilor ilegale.

Ideea generală a proiectului este de a reconstrui, la o scară redusă și în scop educațional, mecanismele care stau la baza platformelor moderne de șah online. În loc să folosească o platformă comercială deja existentă, proiectul își propune să arate concret cum se leagă între ele componentele unei astfel de aplicații: interfața grafică pe care o vede jucătorul, logica de validare a regulilor de șah, serverul care mediază comunicarea dintre cei doi jucători și baza de date care păstrează rezultatele. Astfel, proiectul devine un studiu practic asupra modului în care o aplicație distribuită este construită de la zero.

Tehnologiile principale folosite în proiect sunt următoarele. Pentru partea de client (interfața grafică și logica locală de joc) se utilizează JavaFX, biblioteca grafică standard pentru aplicații desktop scrise în Java. Pentru partea de server se folosește Spring Boot, un cadru de lucru (framework) larg utilizat pentru construirea de aplicații Java care expun servicii web. Comunicarea dintre client și server se realizează prin apeluri REST peste HTTP. Persistența datelor (salvarea partidelor și a mutărilor) se face într-o bază de date relațională MariaDB, care este rulată izolat într-un container Docker. Aceste tehnologii sunt frecvent întâlnite în mediul profesional, ceea ce face ca proiectul să aibă și o valoare formativă orientată către practica industrială.

1.2. Motivarea alegerii temei

Șahul este o temă foarte potrivită pentru un proiect software din mai multe motive. În primul rând, regulile jocului sunt complexe și bine definite: fiecare tip de piesă are propriul mod de deplasare, există situații speciale precum șahul, șah-matul și patul, iar legalitatea unei mutări depinde nu doar de piesa mutată, ci și de starea întregii table. Implementarea acestor reguli obligă programatorul să gândească atent în termeni de stare a jocului, validare și actualizare a stării, ceea ce reprezintă un exercițiu valoros de modelare a unei probleme reale.

În al doilea rând, o partidă de șah între doi jucători presupune o nevoie clară de sincronizare. Cei doi participanți trebuie să vadă aceeași tablă, mutările unuia trebuie să apară pe ecranul celuilalt, iar ordinea mutărilor și a turelor trebuie respectată cu strictețe. Această cerință transformă un simplu joc de șah local într-o aplicație distribuită, în care apar probleme tipice sistemelor client-server: cum se transmit datele, cum se evită aplicarea aceleiași mutări de două ori, cum se detectează finalul partidei pe ambele instanțe de client.

În al treilea rând, arhitectura client-server este unul dintre modelele fundamentale din ingineria software. Aproape orice aplicație modernă (de la rețele sociale la magazine online) se bazează pe separarea dintre un client care prezintă informația și un server care o gestionează. Un joc de șah multiplayer oferă un context concret și ușor de înțeles pentru a exersa această

arhitectură: rolurile sunt clare, fluxul de date este vizibil, iar rezultatul poate fi testat manual de către dezvoltator.

Popularitatea șahului online susține relevanța temei. Conform platformei Chess.com, numărul membrilor înregistrați a depășit pragul de 200 de milioane în aprilie 2025, cu peste 20 de milioane de partide jucate zilnic și aproximativ 85% dintre înregistrările noi provenind din afara Statelor Unite, conform datelor preluate de TechCrunch [1]. Aceeași platformă a anunțat ulterior atingerea pragului de 250 de milioane de membri [2]. Platforma open-source Lichess, dezvoltată de o echipă restrânsă și susținută prin donații, a ajuns să găzduiască milioane de partide pe zi, valorile crescând semnificativ în perioada pandemiei [3]. La nivel de piață, valoarea globală a sectorului de șah a fost estimată la 3,45 miliarde de dolari în 2025, cu o proiecție de creștere până în 2034, conform Fortune Business Insights, citată într-o sinteză statistică recentă [4]. Aceste cifre arată că șahul nu este doar un joc clasic, ci și un domeniu activ din punct de vedere tehnologic, ceea ce face ca un proiect didactic pe această temă să fie atât motivant, cât și ancorat în realitate.

Presupunere / Observație: Statisticile de mai sus au fost preluate din surse online publice și sunt citate în secțiunea de Bibliografie. Valorile se pot modifica în timp; ele sunt folosite aici doar pentru a ilustra amploarea fenomenului șahului online, nu ca date exacte ale prezentului proiect.

1.3. Obiectivele proiectului

Obiectivele proiectului au fost stabilite astfel încât să acopere atât partea vizibilă a aplicației (interfața și jocul propriu-zis), cât și partea de infrastructură (serverul, comunicarea și baza de date). Principalele obiective sunt următoarele:

- Implementarea unei interfețe grafice pentru șah în JavaFX, care să afișeze tabla, piesele, ceasurile, istoricul mutărilor și controalele de joc.
- Implementarea logicii locale de joc, inclusiv reprezentarea tablei, a pozițiilor și a mutărilor, precum și validarea regulilor de șah pe partea de client.
- Realizarea unui server Spring Boot care să gestioneze matchmaking-ul, adică punerea în legătură a doi jucători într-o singură partidă.
- Sincronizarea mutărilor între cei doi clienți, astfel încât mutarea făcută de un jucător să apară pe tabla celuilalt.
- Salvarea partidelor și a mutărilor într-o bază de date MariaDB, la finalul fiecărei partide.
- Implementarea unui timer (ceas de șah) separat pentru fiecare jucător, care scade timpul doar pentru cel aflat la mutare.
- Implementarea funcționalităților de final de joc: remiză, cedare, semnalarea mutărilor ilegale (flag), șah-mat, pat și timp expirat.
- Realizarea unei structuri clare și modulare client-server, cu separarea responsabilităților între interfață, logică, comunicare și persistență.

Atingerea acestor obiective nu urmărește obținerea unei platforme comerciale competitive, ci demonstrarea funcțională a unui sistem complet care integrează toate straturile unei aplicații distribuite. Limitările existente sunt asumate și discutate explicit într-o secțiune dedicată, tocmai pentru a păstra un ton onest și profesional.

2. Stadiu actual

2.1. Soluții existente

Pe piața jocurilor de șah online există deja soluții mature și foarte populare. Cele mai cunoscute platforme sunt Chess.com și Lichess. Chess.com este o platformă comercială cu un număr foarte mare de utilizatori înregistrați, care oferă partide în timp real, antrenament prin puzzle-uri, analiză asistată de motoare de șah și un sistem complex de rating și clasamente. Lichess este o platformă open-source și gratuită, care oferă funcționalități similare (partide rapide, turnee, puzzle-uri, analiză) fără reclame și fără paywall, fiind susținută în principal prin donații.

Pe lângă aceste platforme web, există și numeroase aplicații desktop de șah, multe dintre ele construite în jurul unor motoare de șah puternice precum Stockfish. Astfel de aplicații se concentrează adesea pe jocul împotriva calculatorului, pe analiza partidelor și pe antrenament, iar nu neapărat pe componenta multiplayer online. Motoarele de șah în sine reprezintă o categorie aparte de software, capabile să evalueze poziții la o adâncime foarte mare și să joace la un nivel cu mult superior celui uman.

Funcționalitățile tipice oferite de soluțiile comerciale și de cele open-source includ: conturi de utilizator și profiluri, sistem de rating (de exemplu Elo sau Glicko), matchmaking automat în funcție de nivel, analiză post-partidă, replay-ul partidelor cu navigare prin mutări, salvarea partidelor (adesea în format PGN), mod spectator pentru a urmări partidele altora și chat între jucători. Tabelul de mai jos sintetizează câteva dintre aceste caracteristici și prezența lor în proiectul curent, pentru a evidenția clar diferența de scop.

Funcționalitate	Soluții comerciale / open-source	Proiectul curent
Partide multiplayer în timp real	Da	Da (prin polling REST)
Matchmaking automat	Da, complex (după rating)	Da, simplu (coadă de așteptare)
Conturi și autentificare	Da	Nu (playerId generat local)
Sistem de rating	Da	Nu
Salvare partide în baza de date	Da	Da (MariaDB)
Istoric și replay mutări	Da	Da (navigare locală)
Timer / ceas de joc	Da	Da (per jucător)
Analiză cu motor de șah	Da	Nu
Mod spectator / chat	Da	Nu
Validare completă a regulilor pe server	Da	Nu (validare în principal pe client)

2.2. Limitări ale soluțiilor existente în context educațional

Platformele comerciale și open-source menționate mai sus sunt, din punct de vedere al utilizatorului final, soluții excelente și mature. Totuși, din perspectivă educațională, ele au o limitare importantă: nu arată studentului cum este implementată arhitectura internă. Un utilizator care joacă pe o astfel de platformă vede doar rezultatul final (tabla, mutările, rezultatul partidei), însă mecanismele din spate (modul în care se face matchmaking-ul, cum

se transmit mutările, cum se salvează datele, cum se sincronizează cei doi jucători) rămân ascunse.

Această lipsă de vizibilitate este firească pentru un produs comercial, dar reprezintă un obstacol pentru cineva care vrea să înțeleagă, la nivel de cod, cum funcționează o aplicație client-server. Codul sursă al platformelor comerciale nu este, în general, public, iar chiar și în cazul Lichess, care este open-source, baza de cod este foarte mare și folosește tehnologii și optimizări care depășesc cu mult nevoile unui proiect didactic introductiv.

De aceea, realizarea unei versiuni proprii, chiar dacă mai simplă, este utilă: ea forțează înțelegerea explicită a fiecărui strat. Pentru a construi sincronizarea mutărilor, dezvoltatorul trebuie să decidă concret cum sunt transmise datele și cum se evită dublarea mutărilor. Pentru a salva partidele, trebuie definite entitățile și relațiile din baza de date. Pentru a face matchmaking, trebuie gândit un mecanism de punere în legătură a jucătorilor. Toate aceste decizii devin vizibile și controlabile într-un proiect propriu, spre deosebire de o platformă închisă.

2.3. Diferența dintre proiectul realizat și soluțiile comerciale

Trebuie subliniat clar că proiectul de față nu își propune să concureze cu platformele comerciale și nici să le egaleze în funcționalitate. Scopul nu este obținerea unui produs gata de lansare pe piață, ci înțelegerea aprofundată a modului în care se implementează o aplicație distribuită completă. Proiectul este, prin natura sa, educațional.

Diferența esențială constă în accent. Pe când o platformă comercială pune accent pe scalabilitate, securitate, experiența utilizatorului și monetizare, proiectul curent pune accent pe claritatea arhitecturii și pe acoperirea tuturor straturilor: interfața grafică (UI), regulile de joc, serverul, baza de date, comunicarea și testarea. Faptul că proiectul include funcționalități precum salvarea partidelor, istoricul de mutări, timerul și finalurile speciale demonstrează că, deși este didactic, el atinge un nivel de complexitate suficient pentru a fi reprezentativ pentru o aplicație reală de tip client-server.

3. Implementarea

Acest capitol reprezintă partea centrală a documentației și descrie în detaliu modul în care a fost implementat proiectul. Pornind de la arhitectura generală, se trece treptat prin componenta de client, logica internă a jocului, validarea regulilor, comunicarea client-server, matchmaking-ul, sincronizarea mutărilor, gestionarea timpului, finalurile de joc, componenta de server, endpoint-urile REST, baza de date, testarea, deciziile de proiectare, limitările și posibilele dezvoltări viitoare. Explicațiile se bazează pe descrierea generală a proiectului și pe clasele și metodele furnizate în fișierul de implementare.

3.1. Arhitectura generală a aplicației

Aplicația este construită după modelul client-server, cu o separare clară între trei componente majore: clientul JavaFX, serverul Spring Boot și baza de date MariaDB. Fiecare componentă are responsabilități bine delimitate, iar comunicarea dintre ele se face prin mecanisme standard: apeluri REST între client și server, respectiv acces JPA între server și baza de date.

Clientul JavaFX este aplicația desktop pe care o rulează fiecare jucător. El se ocupă de afișarea interfeței grafice, de tabla de șah, de validarea locală a regulilor, de gestionarea ceasurilor și de istoricul mutărilor. Tot clientul este cel care inițiază comunicarea cu serverul, atât pentru a intra în matchmaking, cât și pentru a trimite mutările și pentru a citi periodic starea jocului.

Serverul Spring Boot joacă rolul de intermediar. El primește cererile de la clienți, gestionează coada de matchmaking, ține în memorie sesiunile active de joc, verifică dacă jucătorul care trimite o mutare este la rândul său și, la finalul partidei, salvează rezultatul și lista de mutări în baza de date. Serverul nu are interfață grafică; el expune doar o serie de endpoint-uri REST pe care clientul le apelează.

Baza de date MariaDB asigură persistența. Spre deosebire de sesiunile active de joc, care sunt ținute doar în memoria serverului, partidele salvate rămân stocate durabil în baza de date și pot fi consultate ulterior prin endpoint-urile de istoric. MariaDB este rulată într-un container Docker, ceea ce simplifică instalarea și izolarea bazei de date față de restul sistemului.

Fluxul general de date poate fi descris pe două direcții. Pe direcția de joc, o mutare făcută de primul client este trimisă către server, care o stochează în sesiunea activă; al doilea client citește această mutare prin polling și o aplică pe propria tablă, realizându-se astfel lanțul Client 1 → Server → Client 2. Pe direcția de persistență, la finalul partidei, clientul anunță serverul despre rezultat, iar serverul scrie datele în baza de date, realizând lanțul Client → Server → Bază de date.

[Figura 1: Diagrama de arhitectură generală. Ar trebui să conțină trei blocuri principale — Client 1 (JavaFX), Server (Spring Boot) și Client 2 (JavaFX) — cu săgeți REST/HTTP între clienți și server, plus un al patrulea bloc (MariaDB în Docker) conectat la server prin JPA. Se recomandă marcarea explicită a porturilor (ex. server pe localhost:8081) și a tipului de comunicare (REST polling).]

3.2. Componenta client

Componenta de client este realizată în JavaFX și este organizată pe mai multe clase, fiecare cu un rol bine definit. Punctul de intrare este clasa MainApp, care extinde Application și

inițializează fereastra principală. La pornire, MainApp afișează ecranul de start (StartView), de unde utilizatorul ajunge în pagina de setări (GameSettingsView). După ce o partidă este găsită, MainApp construiește ecranul de joc prin metoda showGame.

StartView reprezintă ecranul inițial al aplicației și oferă punctul de pornire al interfeței, conducând utilizatorul către zona de setări și de începere a jocului. Clasa LoginView este o componentă auxiliară de interfață pentru o eventuală zonă de autentificare, însă în fluxul actual funcționalitatea principală este concentrată în StartView, GameSettingsView și MainApp, iar LoginView nu este esențială pentru matchmaking, mutări sau sincronizare.

Pagina de setări, implementată în GameSettingsView, permite alegerea timpului de joc (3, 5, 10, 15 sau 30 de minute), a culorii dorite (Alb, Negru sau Random) și activarea opțiunii „Allow illegal moves”, folosită pentru testare și demonstrații. La apăsarea butonului Start Game, clientul trimite o cerere de matchmaking către server. Fragmentul de mai jos prezintă logica esențială declanșată de acest buton.

Fragment din GameSettingsView.java — pornirea jocului și matchmaking-ul

```
startBtn.setOnAction(e -> {
    startBtn.setDisable(true);
    startBtn.setText("Se caută adversar...");

    new Thread(() -> {
        try {
            ChessServerClient serverClient =
                new ChessServerClient("http://localhost:8081");

            String playerId = "player-" + System.currentTimeMillis();
            String response = serverClient.joinGame(playerId);

            if (response.startsWith("Game started")) {
                String gameId = serverClient.extractGameId(response);
                String gameJson = serverClient.getGameState(gameId);
                String assignedColor =
                    serverClient.determinePlayerColor(gameJson, playerId);

                javafx.application.Platform.runLater(() ->
                    mainApp.showGame(selectedTime, assignedColor,
                                    gameId, playerId, allowIllegalMoves));
                return;
            }

            if (response.startsWith("Waiting")) {
                // primul jucător așteaptă adversar -> polling
                while (true) {
                    Thread.sleep(1000);
                    String gameResponse = serverClient.getGameByPlayer(playerId);
                    String gameId =
                        serverClient.extractGameIdFromJson(gameResponse);
                    if (gameId != null) {
                        String assignedColor =
                            serverClient.determinePlayerColor(gameResponse, playerId);
                        javafx.application.Platform.runLater(() ->
                            mainApp.showGame(selectedTime, assignedColor,
```

```

                                gameId, playerId, allowIllegalMoves));
                    break;
                }
            }
        }
    } catch (Exception ex) { ex.printStackTrace(); }
}).start();
});

```

Acest fragment este important pentru că surprinde întregul flux de pornire a unei partide din perspectiva clientului. Mai întâi se generează un identificator local de jucător sub forma „player-” urmat de momentul curent în milisecunde, ceea ce asigură unicitatea în practică. Apoi se apelează metoda `joinGame` a clientului de comunicare, iar răspunsul serverului este interpretat: dacă începe cu „Game started”, înseamnă că s-a format imediat o partidă (jucătorul a fost al doilea în coadă) și se trece direct la ecranul de joc; dacă începe cu „Waiting”, jucătorul este primul și intră într-o buclă de polling, întrebând periodic serverul dacă i s-a găsit un adversar. Întreaga operație rulează pe un fir de execuție separat, pentru a nu bloca interfața grafică, iar trecerea la ecranul de joc se face prin `Platform.runLater`, deoarece actualizarea UI trebuie să se petreacă pe firul JavaFX.

Presupunere / Observație: În acest flux, culoarea efectivă a jucătorului este stabilită de metoda `determinePlayerColor` pe baza stării jocului primite de la server (adică pe baza ordinii de intrare în partidă), nu de opțiunea „Culoare” aleasă în pagina de setări. Prin urmare, în implementarea actuală, selecția de culoare din interfață pare a fi suprascrisă de atribuirea făcută de server (`player1 = alb`, `player2 = negru`). Aceasta este o observație derivată direct din cod.

Ecranul de joc este construit de metoda `showGame` din `MainApp`. Acesta conține, în partea centrală, tabla de șah (`BoardView`, învelită de `BoardWithCoords` pentru afișarea coordonatelor pe margini), iar în partea dreaptă un panou cu titlul partidei, ceasurile celor doi jucători, butoanele de control (`Remiză`, `Cedează`, `Flag`, `Flip`), o casetă de stare (`tură` și `status`) și istoricul mutărilor. Istoricul este afișat într-un tabel cu trei coloane (numărul mutării, mutarea albului și mutarea negrului), folosind clasa auxiliară `MoveRow`, care grupează aceste trei valori pe un singur rând. Sub tabel se găsesc controalele de navigare prin istoric: început, mutarea precedentă, mutarea următoare și final.

Selectarea pieselor și aplicarea mutărilor se realizează în `BoardView`. Conform descrierii sale, această clasă construiește pătratele tablei și piesele, gestionează click-urile utilizatorului, selectarea pieselor, aplicarea mutărilor locale și aplicarea mutărilor primite de la server. Practic, jucătorul dă click pe piesa pe care vrea să o mute și apoi pe pătratul de destinație; `BoardView` colaborează cu `GameState` și `RulesEngine` pentru a valida și aplica mutarea. Tot `BoardView` expune și modul de testare cu mutări ilegale, atunci când opțiunea „Allow illegal moves” a fost activată în setări (prin metoda `setAllowIllegal`).

Un aspect esențial este blocarea tablei atunci când nu este rândul jucătorului local. Această logică este implementată în metoda `updateBoardInputState` din `MainApp`, prezentată mai jos.

Fragment din `MainApp.java` — blocarea tablei în funcție de tură

```
private void updateBoardInputState(BoardView boardView,
                                   GameState state,
                                   String playerColor,
                                   boolean gameOver) {
    if (gameOver) {
        boardView.setDisable(true);
        boardView.setMouseTransparent(true);
        return;
    }

    boolean myTurn = state.getTurn().name().equals(playerColor);

    boardView.setDisable(!myTurn);
    boardView.setMouseTransparent(!myTurn);
}
```

Metoda decide dacă tabla trebuie să accepte interacțiuni. Dacă jocul s-a terminat, tabla este dezactivată complet. Altfel, se compară tura curentă (`state.getTurn()`) cu culoarea jucătorului local (`playerColor`); dacă acestea coincid, este rândul jucătorului și tabla rămâne activă, altfel tabla este dezactivată și devine transparentă la mouse. Astfel, chiar dacă ambele ferestre de client rulează pe același calculator, fiecare instanță se comportă ca un jucător separat, putând muta doar când îi vine rândul. Această metodă este apelată în cadrul rutinei generale de reîmprospătare a interfeței (`refresh`), care actualizează simultan eticheta de tură, statusul, tabelul de mutări și starea butoanelor de navigare.

Tot în `showGame` este definit și comportamentul declanșat după ce o mutare este făcută local, prin callback-ul `setOnMoveMade`. Acesta incrementează contorul de mutări sincronizate local și trimite mutarea către server pe un fir separat, folosind `ChessServerClient.sendMove`. Acest mecanism va fi detaliat în secțiunea dedicată sincronizării mutărilor.

[Figura 2: Diagrama de clase (simplificată) a componentei de client. Ar trebui să cuprindă MainApp, StartView, GameSettingsView, BoardView, BoardWithCoords, MoveRow și ChessServerClient, cu relațiile de utilizare dintre ele (de exemplu MainApp folosește BoardView și ChessServerClient).]

3.3. Logica internă a jocului de șah

Logica internă a jocului este concentrată pe partea de client și este organizată în jurul clasei `GameState`, care reprezintă starea curentă a unei partide. Conform descrierii sale, `GameState` reține tabla, tura curentă, statusul jocului, istoricul pozițiilor și lista de mutări. Este o clasă centrală, deoarece toate mutările modifică această stare, iar interfața grafică se reîmprospătează pe baza ei.

Tabla de șah este modelată la nivel intern de clasa `Board`, care reține piesele pe poziții de tip rând/coloană și oferă metode pentru citirea și modificarea conținutului unei celule. `Board` este folosită de `GameState`, de `RulesEngine` și de `BoardView`, fiind reprezentarea de date asupra căreia operează atât logica de validare, cât și afișarea grafică.

O poziție pe tablă este modelată de clasa `Position`, prin coordonate de tip `row` și `col`. Pozițiile sunt folosite în construcția mutărilor și în logica de validare a deplasării pieselor. O mutare propriu-zisă este modelată de clasa `Move` (versiunea de client), care conține poziția inițială,

poziția finală și eventual informații suplimentare, cum ar fi promovarea. Move din client este folosită de RulesEngine, MoveValidator și MoveApplier.

Tura curentă și culorile sunt gestionate prin enumerarea Color, care definește culorile jucătorilor (de regulă WHITE și BLACK). Color este folosită în GameState, în BoardView și în logica de validare pentru a determina al cui este rândul și ce culoare au piesele. Statusul partidei este descris de enumerarea GameStatus, care definește stările posibile precum joc normal, șah, șah-mat și pat; pe baza acestor stări, interfața afișează mesaje corespunzătoare și oprește jocul în situațiile finale.

Istoricul pozițiilor este gestionat de clasa History, care permite navigarea înainte și înapoi prin mutări. Această funcționalitate stă la baza butoanelor de replay din interfață. Fiecare poziție memorată este reprezentată de un Snapshot, adică o captură a stării jocului la un anumit moment; împreună cu History, snapshot-urile permit reîncărcarea pozițiilor anterioare atunci când utilizatorul navighează prin istoric. Fragmentul de mai jos, extras din MainApp, arată cum se realizează navigarea la o anumită mutare folosind aceste mecanisme.

Fragment din MainApp.java — navigarea în istoric prin snapshot-uri

```
private void jumpToMove(GameState state, BoardView boardView, int targetMoveIndex) {
    while (state.getHistory().canPrev()) {
        state.loadSnapshot(state.getHistory().prev());
    }

    for (int i = 0; i < targetMoveIndex; i++) {
        if (state.getHistory().canNext()) {
            state.loadSnapshot(state.getHistory().next());
        }
    }

    boardView.deselectForExternal();
    boardView.forceRender();
}
```

Metoda jumpToMove derulează istoricul până la o anumită mutare. Mai întâi se revine la poziția inițială prin apeluri repetate la prev() cât timp există o poziție anterioară (canPrev()), apoi se înaintează exact până la indexul dorit folosind next(). Fiecare pas încarcă un snapshot prin loadSnapshot, reconstruind starea tablei. La final, tabla este deselectedată și re-randată. Această metodă este apelată, de exemplu, atunci când utilizatorul dă click pe o celulă din tabelul de istoric, permițând astfel vizualizarea rapidă a unei poziții anterioare, similar cu platformele moderne de șah.

Notăția mutărilor este generată de clasa RomanianSAN, care transformă mutările interne în texte afișabile în istoric, folosind denumiri românești pentru piese. Astfel, mutările care apar în tabelul de istoric (pe coloanele „Alb” și „Negru”) sunt prezentate într-o formă lizibilă pentru utilizatorul român, în loc de simple perechi de coordonate.

3.4. Validarea regulilor de șah

Validarea regulilor de șah este realizată printr-un set de clase specializate, care colaborează pentru a decide dacă o mutare este legală și pentru a actualiza statusul partidei. Coordonatorul acestui proces este RulesEngine, care verifică dacă o mutare este validă, o

aplică asupra stării jocului și actualizează statusul. RulesEngine este, alături de GameState, una dintre clasele cele mai importante pentru logica locală a clientului.

Verificarea propriu-zisă a validității unei mutări este responsabilitatea clasei MoveValidator. Aceasta folosește regulile pieselor, starea tablei și situațiile speciale pentru a decide dacă o mutare este permisă. Odată ce o mutare a fost validată, aplicarea ei efectivă (modificarea pozițiilor pieselor pe tablă) este realizată de MoveApplier, care contribuie la actualizarea stării jocului. Separarea dintre validare (MoveValidator) și aplicare (MoveApplier) menține codul clar și ușor de urmărit.

Regulile specifice fiecărei piese sunt tratate prin PieceMoveResolver, care determină tipul de deplasare permis pentru fiecare piesă, separând astfel regulile specifice pieselor de restul logicii de validare. La nivel și mai fin, fișierele din zona pieces/ definesc comportamentul individual al pieselor (pion, cal, nebun, turn, damă, rege), ceea ce organizează clar logica de deplasare. Pentru situații care depind de pătratele atacate (precum șahul, șah-matul, patul și siguranța regelui), se folosește AttackMap, o clasă care calculează pătratele aflate sub atac. În sfârșit, RulesUtil oferă metode auxiliare folosite în logica regulilor, de exemplu pentru a determina culoarea sau tipul unei piese.

Pe baza acestor clase, aplicația poate distinge între o partidă normală și situațiile speciale. Atunci când regele unui jucător este atacat, statusul devine șah (CHECK); dacă jucătorul nu mai are nicio mutare legală care să scoată regele din atac, statusul devine șah-mat (CHECKMATE); dacă jucătorul nu are nicio mutare legală, dar regele nu este atacat, statusul devine pat (STALEMATE). Aceste stări sunt reflectate în interfață prin mesaje și, în cazul șah-matului și al patului, prin oprirea jocului.

Modul „Allow illegal moves” reprezintă o excepție deliberată de la validare. Atunci când este activat (prin setAllowIllegal pe BoardView), jucătorul poate efectua și mutări care, în mod normal, ar fi respinse. Acest mod este util pentru testare și pentru demonstrarea funcționalității de flag, deoarece permite generarea intenționată a unei situații ilegale care apoi poate fi semnalată. BoardView ține evidența faptului că s-a produs o mutare ilegală (prin metode precum hasIllegalMove și getIllegalMoveColor), informație folosită ulterior de butonul Flag.

Un aspect arhitectural important este că validarea completă a regulilor de șah se face în principal pe client. Serverul verifică mai ales existența jocului și faptul că jucătorul care trimite o mutare este la rând, dar nu reproduce întreaga logică de legalitate șahistică. Această decizie simplifică serverul, însă introduce o limitare clară: un server ideal ar trebui să valideze și legalitatea completă a mutărilor, pentru a preveni manipularea clientului. În forma actuală, un client modificat ar putea, teoretic, să trimită mutări ilegale pe care serverul le-ar accepta atâta timp cât respectă tura. Această limitare este asumată și discutată din nou în secțiunea de limitări.

3.5. Comunicarea client-server

Comunicarea dintre clientul JavaFX și serverul Spring Boot este realizată prin clasa ChessServerClient. Conform descrierii sale, această clasă folosește HttpClient pentru a trimite cereri REST către server, cum ar fi intrarea în matchmaking, obținerea stării jocului, trimiterea mutărilor, finalizarea partidei și citirea istoricului. Clasa conține și metode auxiliare pentru extragerea identicatorului de joc (gameId), determinarea culorii jucătorului și parsarea mutărilor primite din JSON.

Metodele clientului de comunicare, așa cum reies din modul în care sunt apelate în MainApp și GameSettingsView, includ: joinGame (înscrierea în matchmaking, prin POST către /game/join), getGameByPlayer (verificarea dacă un jucător are deja un joc activ, prin GET către /game/player/{playerId}), getGameState (obținerea stării jocului, prin GET către /game/{gameId}), sendMove (trimiterea unei mutări, prin POST către /game/{gameId}/move) și endGame (finalizarea partidei, prin POST către /game/{gameId}/end). Tot din descrierea clasei rezultă că ea oferă și metode pentru citirea istoricului jocurilor și a mutărilor salvate, corespunzătoare endpoint-urilor /game/history și /game/{gameId}/saved-moves.

Presupunere / Observație: Corpurile metodelor din ChessServerClient (de exemplu joinGame, getGameState, sendMove, endGame, extractGameId, determinePlayerColor, extractMovesFromGameJson) nu au fost furnizate ca sursă completă, ci doar descriptiv și prin modul lor de utilizare. Comportamentul lor a fost dedus din apelurile din MainApp și GameSettingsView și din endpoint-urile serverului. Semnăturile exacte și implementarea internă rămân cele din ChessServerClient.java.

Formatul de date folosit în comunicare combină răspunsuri de tip text simplu cu obiecte JSON. Cererile de matchmaking și de finalizare returnează mesaje text scurte (de exemplu „Game started! ID: ...”, „Waiting for opponent...” sau „Game saved with result: ...”), pe care clientul le interpretează prin metode precum extractGameId. În schimb, starea jocului și istoricul sunt returnate sub formă de JSON, pe care clientul îl parsează pentru a extrage informațiile necesare. Pentru trimiterea unei mutări, clientul transmite în corpul cererii un obiect JSON care conține identificatorul jucătorului și coordonatele mutării.

Trimiterea unei mutări pornește din callback-ul setOnMoveMade definit în MainApp. După ce mutarea a fost aplicată local, ea este transmisă serverului prin sendMove, pe un fir separat, ca în fragmentul următor.

Fragment din MainApp.java — trimiterea unei mutări către server

```
boardView.setOnMoveMade(moveInfo -> {
    // mutarea a fost deja aplicată local -> o considerăm sincronizată local
    syncedMoveCount[0]++;

    new Thread(() -> {
        try {
            ChessServerClient serverClient =
                new ChessServerClient("http://localhost:8081");

            String response = serverClient.sendMove(
                gameId, playerId,
                moveInfo.fromRow(), moveInfo.fromCol(),
                moveInfo.toRow(), moveInfo.toCol());

            System.out.println("Mutare trimisă către server: " + response);

            if (!"Move added".equals(response)) {
                System.out.println("Serverul nu a acceptat mutarea: " + response);
            }
        } catch (Exception ex) { ex.printStackTrace(); }
    }).start();
});
```

Acest fragment ilustrează modul în care clientul trimite o mutare. Imediat după aplicarea locală, contorul `syncedMoveCount` este incrementat pentru a marca faptul că mutarea curentă este deja prezentă pe acest client (astfel încât polling-ul să nu o aplice din nou). Apoi, pe un fir separat, se apelează `sendMove` cu identificatorul jocului, identificatorul jucătorului și cele patru coordonate ale mutării. Răspunsul serverului este verificat: dacă nu este exact „Move added”, se afișează un mesaj de avertizare, ceea ce permite depistarea cazurilor în care serverul refuză mutarea (de exemplu „Not your turn”).

Citirea stării jocului se face prin `getGameState`, apelată periodic în bucla de polling. Răspunsul este un JSON care reprezintă sesiunea de joc, din care clientul extrage mutările folosind `extractMovesFromGameJson`; rezultatul este o listă de obiecte `ServerMove`, fiecare cu cele patru coordonate. Mai jos sunt prezentate exemple plauzibile de cereri și răspunsuri JSON, construite pe baza structurilor de date din cod.

Exemplu — corpul cererii POST /game/{gameId}/move (JSON)

```
{
  "playerId": "player-1716800000000",
  "fromRow": 6,
  "fromCol": 4,
  "toRow": 4,
  "toCol": 4
}
```

Acest obiect corespunde clasei `MoveDTO` de pe server, care conține `playerId` și coordonatele `fromRow`, `fromCol`, `toRow`, `toCol`. El descrie, de exemplu, o mutare de pion de două pătrate la deschidere. Răspunsul serverului este un text scurt: „Move added” în caz de succes sau un mesaj de eroare.

Exemplu — răspuns GET /game/{gameId} (stare joc, JSON plauzibil)

```
{
  "gameId": "f1c2e8a0-7b3d-4c9a-9e21-aa55bb66cc77",
  "player1": { "id": "player-1716800000000" },
  "player2": { "id": "player-1716800005321" },
  "moves": [
    { "fromRow": 6, "fromCol": 4, "toRow": 4, "toCol": 4 },
    { "fromRow": 1, "fromCol": 4, "toRow": 3, "toCol": 4 }
  ],
  "currentTurn": "WHITE",
  "whiteTimeMs": 598500,
  "blackTimeMs": 599200
}
```

Acest JSON corespunde structurii clasei `GameSession` de pe server (identificatorul partidei, cei doi jucători, lista de mutări, tura curentă și timpii rămași pentru fiecare culoare). Clientul îl folosește în două scopuri: pentru a determina culoarea jucătorului (comparând `playerId` cu identificatorii `player1` și `player2`) și pentru a extrage lista de mutări, pe care apoi o compară cu mutările deja sincronizate local. Câmpurile au fost deduse din metodele de tip getter expuse de `GameSession`; valorile concrete sunt ilustrative.

Presupunere / Observație: Exemplele JSON de mai sus (în special structura stării jocului) sunt plauzibile și derivate din câmpurile și metodele claselor `GameSession`, `Player`, `Move` și `MoveDTO`. Ele nu reprezintă capturi reale de trafic, ci reconstituiri bazate pe cod, oferite pentru a ilustra forma datelor schimbate între client și server.

3.6. Matchmaking

Matchmaking-ul este mecanismul prin care doi jucători sunt puși în legătură într-o singură partidă. În acest proiect, el este simplu și se bazează pe o coadă de așteptare. Primul jucător care apasă Start Game și trimite o cerere de înscriere este pus într-o coadă; el nu are încă un adversar și primește răspunsul „Waiting for opponent...”. Când al doilea jucător intră, serverul îl ia pe primul din coadă, creează o sesiune de joc nouă și returnează confirmarea că partida a început.

Pe server, logica de matchmaking este implementată în `MatchmakingService`, care folosește două structuri de date: o coadă de jucători în așteptare (`waitingPlayers`) și o hartă a jocurilor active (`activeGames`), indexată după `gameId`. Fragmentul de mai jos prezintă metoda `joinGame`.

Fragment din `MatchmakingService.java` — intrarea în matchmaking

```
private Queue<Player> waitingPlayers = new LinkedList<>();
private Map<String, GameSession> activeGames = new HashMap<>();

public GameSession joinGame(Player player) {
    if (waitingPlayers.isEmpty()) {
        waitingPlayers.add(player);
        return null;           // primul jucător -> așteaptă
    }

    Player opponent = waitingPlayers.poll();
    GameSession session = new GameSession(opponent, player);
    activeGames.put(session.getId(), session);
    return session;           // al doilea jucător -> creează partida
}
```

Metoda `joinGame` ilustrează esența matchmaking-ului. Dacă nu există niciun jucător în așteptare, jucătorul curent este adăugat în coadă și metoda returnează `null`, semnalând astfel starea de așteptare (controllerul va răspunde „Waiting for opponent...”). Dacă există deja un jucător în coadă, acesta este scos (`poll`), iar din cei doi se creează o nouă sesiune de joc, care este adăugată în harta jocurilor active. Important de remarcat este ordinea parametrilor: `new GameSession(opponent, player)`, unde `opponent` este jucătorul care aștepta deja. Cum în `GameSession` primul parametru devine `player1`, rezultă că primul jucător care a intrat (cel care aștepta) este `player1`, iar al doilea este `player2`.

Această ordine confirmă o regulă enunțată în descrierea proiectului: `player1` este jucătorul cu piesele albe, iar `player2` este jucătorul cu piesele negre. Practic, primul jucător care intră în matchmaking primește piesele albe, iar al doilea primește piesele negre. Verificarea turei se bazează tot pe această asociere, prin metoda `isPlayersTurn` din `GameSession`.

Endpoint-urile implicate în matchmaking sunt `POST /game/join` (înscrierea unui jucător) și `GET /game/player/{playerId}` (verificarea dacă un jucător are deja un joc activ). Primul jucător,

aflat în starea de așteptare, nu primește imediat un gameId; de aceea el folosește polling: apelează periodic `getGameByPlayer` (care interoghează endpoint-ul `/game/player/{playerId}`) până când serverul îi returnează o sesiune de joc, moment în care extrage gameId-ul și intră în ecranul de joc. Al doilea jucător, în schimb, primește direct confirmarea „Game started” și gameId-ul, deci nu mai are nevoie de polling pentru a începe.

[Figura 3: Diagrama de secvență a matchmaking-ului. Ar trebui să arate: Client A trimite POST `/game/join` și primește „Waiting”; Client A face polling pe GET `/game/player/{id}`; Client B trimite POST `/game/join` și primește „Game started! ID”; serverul creează `GameSession`; la următorul polling, Client A primește sesiunea și intră în joc.]

3.7. Sincronizarea mutărilor

Sincronizarea mutărilor între cei doi clienți este una dintre cele mai importante componente ale proiectului, deoarece ea asigură că ambii jucători văd aceeași partidă. Mecanismul se bazează pe combinarea aplicării locale a mutărilor cu citirea periodică (polling) a stării jocului de la server.

Fluxul este următorul. Atunci când jucătorul local face o mutare, aceasta este mai întâi aplicată local pe propria tablă (prin logica din `BoardView` și `RulesEngine`) și abia apoi este trimisă către server prin `sendMove`. Serverul o stochează în sesiunea activă. Celălalt client nu este notificat direct, ci află despre mutare prin polling: la intervale scurte, el cere starea jocului, compară numărul de mutări pe care le are deja sincronizate local cu numărul de mutări existente pe server și aplică doar mutările lipsă.

Pentru a evita aplicarea aceleiași mutări de două ori, se folosește contorul `syncedMoveCount`. Acesta reține câte mutări au fost deja reflectate pe clientul local. Când jucătorul mută local, contorul este incrementat imediat (mutarea proprie nu mai trebuie reaplicată din polling). Când polling-ul găsește pe server mai multe mutări decât valoarea contorului, sunt aplicate doar cele suplimentare. Fragmentul de mai jos, extras din bucla de polling din `MainApp`, arată acest mecanism.

Fragment din `MainApp.java` — aplicarea mutărilor remote prin polling

```
String gameJson = serverClient.getState(gameId);
// ... verificări dacă jocul mai există ...

List<ChessServerClient.ServerMove> serverMoves =
    serverClient.extractMovesFromGameJson(gameJson);

if (serverMoves.size() > syncedMoveCount[0]) {
    List<ChessServerClient.ServerMove> movesToApply =
        new ArrayList<>(serverMoves.subList(
            syncedMoveCount[0], serverMoves.size()));

    Platform.runLater(() -> {
        for (ChessServerClient.ServerMove remoteMove : movesToApply) {
            boolean applied = boardView.applyRemoteMove(
                remoteMove.fromRow(), remoteMove.fromCol(),
                remoteMove.toRow(),  remoteMove.toCol());
            if (applied) {
```

```

        syncedMoveCount[0]++;
        if (refreshRef[0] != null) refreshRef[0].run();
    } else {
        System.out.println("Nu s-a putut aplica mutarea remote: ...");
    }
}
});
}

```

Acest fragment este nucleul sincronizării. Se obține starea jocului de la server și se extrage lista de mutări (serverMoves). Dacă pe server există mai multe mutări decât a sincronizat clientul local (serverMoves.size() > syncedMoveCount), se calculează submulțimea de mutări noi prin subList, pornind de la indexul syncedMoveCount. Aceste mutări sunt apoi aplicate pe tablă prin applyRemoteMove, pe firul JavaFX (Platform.runLater), pentru a respecta regula că interfața se modifică doar pe firul de UI. Fiecare mutare aplicată cu succes incrementează contorul și declanșează reîmprospătarea interfeței. Dacă o mutare nu poate fi aplicată, se afișează un mesaj de diagnostic. Datorită acestui mecanism, atunci când un jucător mută pe un client, celălalt client vede actualizarea pe propria tablă după aproximativ o secundă (intervalul de polling).

Distincția dintre mutările proprii și cele remote este esențială. Mutările proprii sunt aplicate direct la momentul interacțiunii și marcate prin incrementarea contorului în callback-ul setOnMoveMade; ele nu mai trebuie reaplicate. Mutările adversarului sosesc prin polling și sunt aplicate prin applyRemoteMove. În modul de testare „Allow illegal moves”, mecanismul de sincronizare funcționează la fel din punct de vedere al transmiterii coordonatelor; diferența este că validarea locală este relaxată, astfel încât și mutările care în mod normal ar fi ilegale pot fi transmise și aplicate, ceea ce este util pentru demonstrarea funcționalității de flag.

3.8. Timer-ul de joc

Fiecare jucător dispune de propriul ceas de joc, gestionat pe partea de client prin clasa Timeline din JavaFX. La intrarea în ecranul de joc, timpul total (în secunde) este calculat din minutele alese în setări, iar pentru fiecare culoare se inițializează o proprietate de tip IntegerProperty. Esența logicii constă în faptul că timpul scade doar pentru jucătorul aflat la mutare.

Timeline-ul rulează un KeyFrame la fiecare secundă. La fiecare tic, se verifică a cui este tura curentă (state.getTurn()) și se scade o secundă din timpul jucătorului corespunzător. Când tura se schimbă, începe să scadă timpul celui alt jucător. Dacă timpul ajunge la zero, partida se termină: ceasul se oprește, tabla se blochează și este afișat câștigătorul. Fragmentul de mai jos prezintă logica pentru jucătorul cu piesele albe.

Fragment din MainApp.java — ceasul de joc (Timeline)

```

timerRef[0] = new Timeline(
    new KeyFrame(Duration.seconds(1), e -> {
        if (gameOverByTime[0]) return;

        if (state.getTurn().name().equals("WHITE")) {
            whiteTime.set(whiteTime.get() - 1);

```

```

        if (whiteTime.get() <= 0) {
            whiteTime.set(0);
            whiteClock.setText(formatTime(whiteTime.get()));
            gameOverByTime[0] = true;
            timerRef[0].stop();

            boardView.setDisable(true);
            boardView.setMouseTransparent(true);

            statusLabel.setText("Negru a câștigat la timp!");
            // ... afișare alertă "Timp expirat" ...
            return;
        }
        whiteClock.setText(formatTime(whiteTime.get()));
    } else {
        // simetric pentru negru ...
    }
})
);
timerRef[0].setCycleCount(Timeline.INDEFINITE);
timerRef[0].play();

```

Fragmentul arată cum funcționează ceasul. Indicatorul `gameOverByTime` previne orice acțiune după ce jocul s-a terminat. Cât timp este rândul albului, la fiecare secundă se decrementează `whiteTime` și se actualizează eticheta corespunzătoare. Dacă `whiteTime` ajunge la zero, jocul este marcat ca terminat, `Timeline`-ul este oprit, tabla este dezactivată, iar mesajul de stare anunță că negrul a câștigat la timp, însoțit de o alertă. Pentru jucătorul negru există o ramură simetrică. Decrementarea selectivă (doar pentru jucătorul aflat la mutare) este asigurată de verificarea turei la fiecare tic.

Oprirea ceasului nu se produce doar la expirarea timpului, ci în toate situațiile de final de joc. La remiză, la cedare și la flag, este apelată metoda `finishLocalGame`, care setează `gameOverByTime`, oprește `Timeline`-ul, blochează tabla și actualizează mesajul de stare. La fel se întâmplă și la șah-mat sau pat, detectate în rutina de `refresh` pe baza statusului din `GameState`. În plus, finalul jocului este propagat și către celălalt client: după ce o partidă este finalizată, serverul o elimină din lista de jocuri active, iar bucla de polling a clientului advers detectează că jocul nu mai există, oprind la rândul ei ceasul și blocând tabla.

3.9. Funcționalități de final de joc

Aplicația implementează mai multe tipuri de final de joc: remiza, cedarea, semnalarea mutărilor ilegale (flag), șah-matul, patul și timpul expirat. Pentru fiecare, comportamentul urmează un tipar comun pe client (oprirea ceasului și blocarea tablei prin `finishLocalGame`), urmat, acolo unde este cazul, de notificarea serverului și de salvarea în baza de date.

3.9.1. Remiza

Remiza este declanșată de butonul „Remiză”. La apăsare, se cere confirmarea utilizatorului; dacă acesta confirmă, se apelează `finishLocalGame` cu un mesaj de remiză, oprind ceasul și blocând tabla. Apoi, pe un fir separat, clientul trimite serverului rezultatul „DRAW” prin `endGame`. Serverul preia sesiunea, o salvează în baza de date cu rezultatul DRAW și o

elimină din lista de jocuri active. Celălalt client observă prin polling că jocul nu mai există și își oprește, la rândul lui, ceasul, blocând tabla.

3.9.2. Cedarea

Cedarea este declanșată de butonul „Cedează”, tot cu confirmare. Spre deosebire de remiză, cedarea stabilește un câștigător: dacă jucătorul local are piesele albe și cedează, câștigă negrul (rezultat „BLACK_WIN”); dacă are piesele negre, câștigă albul (rezultat „WHITE_WIN”). Pe client se apelează `finishLocalGame` cu mesajul corespunzător, iar pe fir separat se trimite serverului rezultatul prin `endGame`. Serverul salvează partida și o elimină din jocurile active, propagând finalul către adversar prin polling.

3.9.3. Flag pentru mutări ilegale

Funcționalitatea de flag permite semnalarea unei mutări ilegale, fiind utilă mai ales în combinație cu modul „Allow illegal moves”. La apăsarea butonului „Flag”, clientul verifică dacă există o mutare ilegală semnalată (prin `hasIllegalMove`). Dacă da, determină culoarea celui care a făcut mutarea ilegală (`getIllegalMoveColor`) și stabilește câștigătorul opus: dacă mutarea ilegală a fost a albului, câștigă negrul, și invers. Fragmentul de mai jos prezintă această logică.

Fragment din `MainApp.java` — semnalarea unei mutări ilegale (flag)

```
flagBtn.setOnAction(e -> {
    if (boardView.hasIllegalMove()) {
        String illegalColor = boardView.getIllegalMoveColor();
        String winner, resultCode;

        if ("WHITE".equals(illegalColor)) {
            winner = "Negru"; resultCode = "BLACK_WIN";
        } else {
            winner = "Alb";   resultCode = "WHITE_WIN";
        }

        finishLocalGame(boardView, timerRef, gameOverByTime,
            statusLabel, winner + " câștigă prin flag!", "#F2C14E");

        new Thread(() -> {
            ChessServerClient serverClient =
                new ChessServerClient("http://localhost:8081");
            serverClient.endGame(gameId, resultCode);
        }).start();
        // ... afișare alertă cu rezultatul ...
    } else {
        new Alert(Alert.AlertType.INFORMATION,
            "Nu există mutare ilegală de semnalat.").showAndWait();
    }
});
```

Codul arată cum, în prezența unei mutări ilegale, se stabilește câștigătorul opus celui care a comis-o, se oprește jocul local prin `finishLocalGame` și se trimite serverului rezultatul (BLACK_WIN sau WHITE_WIN) prin `endGame`. Dacă nu există nicio mutare ilegală de semnalat, utilizatorul primește un mesaj informativ. Această funcționalitate transformă o

situație de testare (o mutare ilegală permisă în mod demo) într-un final de joc clar, demonstrând în același timp că logica de final și salvarea în baza de date funcționează.

3.9.4. Șah-mat, pat și timp expirat

Șah-matul și patul sunt detectate local, în rutina de refresh, pe baza statusului calculat de GameState și RulesEngine. Când statusul devine CHECKMATE sau STALEMATE, jocul este marcat ca terminat, Timeline-ul este oprit și tabla este blocată. Timpul expirat este tratat direct în logica ceasului, așa cum s-a descris în secțiunea anterioară: când timpul unui jucător ajunge la zero, câștigă adversarul. În toate aceste cazuri locale, tabla se blochează și ceasul se oprește; pentru ca rezultatul să fie persistat, este nevoie ca finalul să fie comunicat serverului prin endGame (de exemplu prin butoanele dedicate), iar serverul să salveze partida și să o elimine din jocurile active.

Tabelul de mai jos sintetizează, pentru fiecare tip de final, ce se întâmplă pe client, ce se trimite la server și cum află celălalt client despre finalul partidei.

Tip de final	Pe client	Trimis la server	Cum află adversarul
Remiză	finishLocalGame, confirmare, oprire ceas, blocare tablă	DRAW (endGame)	Polling — jocul nu mai e activ
Cedare	finishLocalGame, câștigător opus, oprire ceas	BLACK_WIN / WHITE_WIN	Polling — jocul nu mai e activ
Flag (mutare ilegală)	Câștigător opus celui ilegal, finishLocalGame	BLACK_WIN / WHITE_WIN	Polling — jocul nu mai e activ
Șah-mat	Detectat din status, oprire ceas, blocare tablă	Prin endGame (la finalizare)	Polling — jocul nu mai e activ
Pat	Detectat din status, oprire ceas, blocare tablă	Prin endGame (la finalizare)	Polling — jocul nu mai e activ
Timp expirat	Ceasul ajunge la 0, câștigă adversarul	Prin endGame (la finalizare)	Polling — jocul nu mai e activ

Presupunere / Observație: Pentru șah-mat, pat și timp expirat, codul furnizat arată clar oprirea locală a jocului (blocarea tablei și a ceasului). Momentul exact și forma apelului către server pentru aceste cazuri specifice nu apar explicit în fragmentele de cod disponibile pentru toate situațiile; s-a presupus, pe baza tiparului din remiză/cedare/flag, că finalizarea pe server se face tot prin endGame cu rezultatul corespunzător.

3.10. Componenta server

Serverul este construit cu Spring Boot, un framework care simplifică crearea de aplicații Java ce expun servicii web. Punctul de intrare este clasa `ChessServerApplication`, care pornește serverul și inițializează contextul Spring, făcând disponibile controller-ele, serviciile și repository-urile. Arhitectura serverului respectă o separare clasică pe straturi: un controller care expune endpoint-urile REST, servicii care conțin logica de aplicație și repository-uri care se ocupă de accesul la baza de date.

Controllerul principal este `GameController`, adnotat cu `@RestController` și mapat pe ruta `/game`. El primește dependențele de care are nevoie (`GameService`, `MatchmakingService`, `DatabaseService` și `WebSocketNotificationService`) prin injecție de constructor și expune endpoint-urile de joc. Fragmentul de mai jos prezintă două dintre cele mai importante metode ale sale: înscrierea în matchmaking și primirea unei mutări.

Fragment din `GameController.java` — `join` și `move`

```
@PostMapping("/join")
public String joinGame(@RequestBody Player player) {
    GameSession session = matchmakingService.joinGame(player);
    if (session == null) {
        return "Waiting for opponent...";
    }
    websocketNotificationService.notifyGameUpdate(
        session.getGameId(), "Game started: " + session.getGameId());
    return "Game started! ID: " + session.getGameId();
}

@PostMapping("/{gameId}/move")
public String makeMove(@PathVariable String gameId, @RequestBody MoveDTO moveDTO) {
    Move move = new Move();
    move.setFromRow(moveDTO.fromRow); move.setFromCol(moveDTO.fromCol);
    move.setToRow(moveDTO.toRow);      move.setToCol(moveDTO.toCol);

    String result = matchmakingService.addMoveToGame(gameId, move, moveDTO.playerId);

    if (result.equals("Move added")) {
        websocketNotificationService.notifyGameUpdate(
            gameId, "Move added by " + moveDTO.playerId);
    }
    return result;
}
```

Acest fragment arată cum controllerul delegă munca către servicii. La `/join`, controllerul apelează `matchmakingService.joinGame`; dacă rezultatul este null (jucătorul a intrat în coadă), răspunde „Waiting for opponent...”, altfel răspunde cu „Game started! ID: ” și identificatorul partidei. La `/gameId/move`, controllerul construiește un obiect `Move` din datele primite (`MoveDTO`), îl trimite serviciului de matchmaking pentru a fi adăugat în sesiune și returnează rezultatul (de exemplu „Move added” sau „Not your turn”). În ambele cazuri, dacă operația reușește, se apelează și serviciul de notificare `WebSocket` — element pregătit pentru o sincronizare viitoare mai rapidă.

Logica de matchmaking și de gestionare a mutărilor se află în `MatchmakingService`. Pe lângă `joinGame` (descrișă în secțiunea de matchmaking), această clasă oferă `getGame` (caută o sesiune după gameId), `getGameByPlayerId` (caută sesiunea în care apare un anumit jucător), `addMoveToGame` (adaugă o mutare după verificarea turei) și `removeGame` (elimină o sesiune). Fragmentul de mai jos prezintă adăugarea unei mutări, care include verificarea turei.

Fragment din `MatchmakingService.java` — adăugarea unei mutări cu verificarea turei

```
public String addMoveToGame(String gameId, Move move, String playerId) {
    GameSession session = activeGames.get(gameId);

    if (session == null) {
        return "Game not found";
    }
    if (!session.isPlayersTurn(playerId)) {
        return "Not your turn";
    }

    session.addMove(move);
    return "Move added";
}
```

Metoda `addMoveToGame` verifică mai întâi existența sesiunii; dacă nu o găsește, returnează „Game not found”. Apoi verifică, prin `isPlayersTurn`, dacă jucătorul care trimite mutarea este într-adevăr la rând; dacă nu, returnează „Not your turn”, refuzând mutarea. Doar dacă ambele condiții sunt îndeplinite, mutarea este adăugată în sesiune prin `addMove` (care și schimbă tura). Aceasta este, de fapt, principala verificare pe care o face serverul: el controlează ordinea turelor, dar nu și legalitatea șahistică completă a mutării.

Sesiunea de joc este reprezentată de `GameSession`, care reține identificatorul partidei, cei doi jucători, lista de mutări, tura curentă și timpii rămași pentru fiecare culoare. Tot aici se află logica de verificare a turei și de comutare a acesteia.

Fragment din `GameSession.java` — tură, adăugare mutare și actualizarea timpului

```
public boolean isPlayersTurn(String playerId) {
    if (currentTurn.equals("WHITE")) {
        return player1.getId().equals(playerId);
    } else {
        return player2.getId().equals(playerId);
    }
}

public void addMove(Move move) {
    updateTime();
    moves.add(move);
    switchTurn();
    lastMoveTimestamp = System.currentTimeMillis();
}

private void switchTurn() {
    currentTurn = currentTurn.equals("WHITE") ? "BLACK" : "WHITE";
}
```

Acest fragment confirmă asocierea dintre culori și jucători: când e rândul albului, doar player1 poate muta, iar când e rândul negrului, doar player2. La adăugarea unei mutări, se actualizează mai întâi timpul (scăzând din timpul jucătorului aflat la mutare intervalul scurs de la ultima mutare), se adaugă mutarea în listă, se comută tura și se reține momentul mutării. Astfel, serverul ține și el o evidență a timpului, pe lângă ceasul afișat pe client.

Clasa Player modelează un jucător pe server, conținând în implementarea actuală identificatorul jucătorului; ea este folosită în matchmaking și în GameSession pentru asocierea celor doi participanți. Clasa Move (versiunea de server) modelează o mutare primită și stocată în sesiunea activă, conținând coordonatele fromRow, fromCol, toRow și toCol; ea este folosită de GameSession, MatchmakingService și GameController pentru sincronizarea mutărilor.

Persistența este realizată de DatabaseService, care folosește două repository-uri JPA (GameRepository și MoveRepository) pentru a salva și citi date. GameService rămâne un serviciu auxiliar, util pentru o listă generală de mutări și pentru resetarea jocului, în timp ce logica multiplayer principală se află în MatchmakingService. În sfârșit, WebSocketConfig și WebSocketNotificationService reprezintă infrastructura pregătită pentru o comunicare în timp real: serverul apelează deja serviciul de notificare la începutul jocului, la mutări și la finalizare, însă clientul folosește în prezent polling REST. Aceste două clase pot fi descrise ca o direcție de dezvoltare viitoare, care ar permite înlocuirea polling-ului cu notificări live.

[Figura 4: Diagrama pe straturi a serverului Spring Boot. Ar trebui să arate stratul Controller (GameController), stratul Service (MatchmakingService, DatabaseService, GameService, WebSocketNotificationService), stratul Repository (GameRepository, MoveRepository) și stratul de persistență (MariaDB), cu modelele GameSession, Player, Move și entitățile GameEntity, MoveEntity.]

3.11. Endpoint-uri REST

Serverul expune mai multe endpoint-uri REST, toate sub prefixul /game, definite în GameController. Tabelul de mai jos rezumă endpoint-urile, metoda HTTP, scopul, parametrii și locul în care sunt folosite în client. Pentru lizibilitate, exemplele de cerere și de răspuns sunt prezentate separat, sub tabel, deoarece conțin obiecte JSON care nu ar încăpea confortabil într-o coloană.

Endpoint	Metodă	Scop	Parametri	Folosit în client
/game/test	GET	Verifică dacă serverul funcționează	—	Verificare manuală
/game/join	POST	Înscrie un jucător în matchmaking	Body: Player (id)	joinGame (Start Game)
/game/player/{playerId}	GET	Verifică dacă jucătorul are deja un joc activ	Path: playerId	getGameByPlayer (polling)
/game/{gameId}	GET	Returnează starea jocului activ	Path: gameId	getGameState (polling)
/game/{gameId}/move	POST	Primește o mutare de la client	Path: gameId; Body: MoveDTO	sendMove
/game/{gameId}/end	POST	Finalizează și salvează partida	Path: gameId; Query:	endGame

Endpoint	Metodă	Scop	Parametri	Folosit în client
			result	
/game/history	GET	Returnează istoricul jocurilor salvate	—	Citire istoric
/game/{gameId}/saved-moves	GET	Returnează mutările salvate ale unei partide	Path: gameId	Citire mutări salvate
/game/moves	GET	Returnează lista generală de mutări (auxiliar)	—	(neutilizat principal)
/game/reset	POST	Resetează jocul (auxiliar)	—	(neutilizat principal)

Ultimele două endpoint-uri (/game/moves și /game/reset) sunt deservite de GameService și apar în controller, dar nu fac parte din fluxul principal multiplayer; ele sunt utile pentru testare sau pentru o gestionare simplă a mutărilor. Mai jos sunt prezentate exemple de cerere și răspuns pentru endpoint-urile centrale.

/game/test — GET (verificare server)

Cerere: GET http://localhost:8081/game/test
 Răspuns: Server merge!

/game/join — POST (înscriere în matchmaking)

Cerere (body JSON):

```
{ "id": "player-1716800000000" }
```

 Răspuns posibil 1 (primul jucător): Waiting for opponent...
 Răspuns posibil 2 (al doilea jucător): Game started! ID: f1c2e8a0-7b3d-4c9a-9e21-aa55bb66cc77

/game/{gameId}/move — POST (trimitere mutare)

Cerere (body JSON):

```
{ "playerId": "player-1716800000000",  
  "fromRow": 6, "fromCol": 4, "toRow": 4, "toCol": 4 }
```

 Răspuns posibil: Move added | Not your turn | Game not found

/game/{gameId}/end — POST (finalizare partidă)

Cerere: POST http://localhost:8081/game/{gameId}/end?result=DRAW
 Răspuns: Game saved with result: DRAW
 (sau "Game not found" dacă sesiunea nu mai există)

/game/history — GET (răspuns JSON plauzibil)

```
[  
  {  
    "gameId": "f1c2e8a0-7b3d-4c9a-9e21-aa55bb66cc77",  
    "player1": "player-1716800000000",  
    "player2": "player-1716800005321",  
    "result": "WHITE_WIN"
```

```
}
]
```

/game/{gameId}/saved-moves — GET (răspuns JSON plauzibil)

```
[
  { "gameId": "f1c2e8a0-...", "moveNumber": 1,
    "fromRow": 6, "fromCol": 4, "toRow": 4, "toCol": 4,
    "playerId": "player-1716800000000" },
  { "gameId": "f1c2e8a0-...", "moveNumber": 2,
    "fromRow": 1, "fromCol": 4, "toRow": 3, "toCol": 4,
    "playerId": "player-1716800005321" }
]
```

Presupunere / Observație: Răspunsurile JSON pentru /game/history și /game/{gameId}/saved-moves au fost reconstituite pe baza câmpurilor descrise pentru GameEntity și MoveEntity. Forma exactă a serializării depinde de configurarea Spring/Jackson din proiect; valorile sunt ilustrative.

3.12. Baza de date

Pentru persistența datelor, proiectul folosește o bază de date relațională MariaDB, rulată într-un container Docker. Rularea în Docker are avantajul izolării și al ușurinței de instalare: baza de date poate fi pornită cu o singură comandă, fără a instala manual un server de baze de date pe sistemul gazdă, iar configurația rămâne reproductibilă. Serverul Spring Boot se conectează la această bază de date prin JPA (Java Persistence API).

Salvarea propriu-zisă este coordonată de DatabaseService, care la finalul unei partide creează o entitate de tip GameEntity pentru partidă și câte o entitate MoveEntity pentru fiecare mutare. Fragmentul de mai jos prezintă metoda saveGame.

Fragment din DatabaseService.java — salvarea partidei și a mutărilor

```
public void saveGame(GameSession session, String result) {
    GameEntity gameEntity = new GameEntity(
        session.getGameId(),
        session.getPlayer1().getId(),
        session.getPlayer2().getId(),
        result);
    gameRepository.save(gameEntity);

    int moveNumber = 1;
    for (Move move : session.getMoves()) {
        String playerId = (moveNumber % 2 == 1)
            ? session.getPlayer1().getId()
            : session.getPlayer2().getId();

        MoveEntity moveEntity = new MoveEntity(
            session.getGameId(), moveNumber,
            move.getFromRow(), move.getFromCol(),
            move.getToRow(), move.getToCol(),
            playerId);
    }
}
```

```

        moveRepository.save(moveEntity);
        moveNumber++;
    }
}

```

Metoda `saveGame` ilustrează clar modelul de persistență. Mai întâi se salvează partida ca `GameEntity`, reținând `gameId`, identificatorii celor doi jucători și rezultatul. Apoi se parcurge lista de mutări din sesiune și, pentru fiecare, se creează un `MoveEntity` cu numărul mutării, coordonatele și identificatorul jucătorului care a efectuat-o. Atribuirea jucătorului se face pe baza parității numărului mutării: mutările impare aparțin lui `player1` (alb), iar cele pare lui `player2` (negru). Aceasta corespunde alternanței turelor într-o partidă de șah.

Accesul la date este abstractizat prin două repository-uri JPA. `GameRepository` este interfața pentru tabela de jocuri salvate, extinzând mecanismele Spring Data JPA și permițând operații de salvare, căutare și citire pentru `GameEntity`. `MoveRepository` este interfața pentru mutările salvate, folosită pentru salvarea mutărilor unei partide și pentru obținerea mutărilor asociate unui anumit `gameId` (prin metoda `findByGameIdOrderByMoveNumberAsc`, care le returnează ordonate după numărul mutării). Aceste repository-uri permit ca `DatabaseService` să nu scrie cod SQL explicit.

Tabelele de mai jos prezintă câmpurile celor două entități, așa cum reies din descrierile și constructorii folosiți în cod.

Entitatea `GameEntity` (partidă salvată)

Câmp	Tip (presupus)	Descriere
<code>gameId</code>	String	Identificatorul unic al partidei (UUID)
<code>player1</code>	String	Identificatorul jucătorului alb
<code>player2</code>	String	Identificatorul jucătorului negru
<code>result</code>	String	Rezultatul (DRAW, WHITE_WIN, BLACK_WIN)

Entitatea `MoveEntity` (mutare salvată)

Câmp	Tip (presupus)	Descriere
<code>gameId</code>	String	Identificatorul partidei a căreia îi aparține mutarea
<code>moveNumber</code>	int	Numărul mutării în cadrul partidei
<code>fromRow</code>	int	Rândul de plecare
<code>fromCol</code>	int	Coloana de plecare
<code>toRow</code>	int	Rândul de destinație
<code>toCol</code>	int	Coloana de destinație
<code>playerId</code>	String	Identificatorul jucătorului care a mutat

Presupunere / Observație: Tipurile coloanelor (String, int) sunt presupuse pe baza valorilor folosite în constructorii `GameEntity` și `MoveEntity`. De asemenea, prezența unei chei primare proprii (de exemplu un câmp `id` auto-generat pentru `MoveEntity`) este probabilă într-o entitate JPA, dar nu este vizibilă explicit în fragmentele furnizate; ea a fost omisă pentru a nu inventa câmpuri.

[Figura 5: Schema logică a bazei de date. Ar trebui să prezinte două tabele — GAMES (gameId PK, player1, player2, result) și MOVES (gameId, moveNumber, fromRow, fromCol, toRow, toCol, playerId) — cu o relație unu-la-mai-mulți între o partidă și mutările sale (un GAMES are mai multe MOVES, legate prin gameId).]

3.13. Testare

Testarea aplicației urmărește verificarea întregului flux, de la pornirea infrastructurii până la salvarea partidelor în baza de date. Având în vedere că sincronizarea se face între doi clienți, multe dintre teste presupun rularea a două instanțe de client, eventual pe același calculator, fiecare comportându-se ca un jucător separat. Strategia generală este una de testare funcțională manuală, pas cu pas, completată de verificarea endpoint-urilor REST și a conținutului bazei de date.

Pașii de pregătire a mediului sunt: pornirea bazei de date MariaDB în Docker, pornirea serverului Spring Boot și verificarea că serverul răspunde, prin accesarea endpoint-ului /game/test (care trebuie să returneze „Server merge!”). Abia după ce aceste condiții sunt îndeplinite se pornesc clienții. Primul client intră în matchmaking și ar trebui să primească starea de așteptare; al doilea client, la pornire, ar trebui să formeze partida, ambii clienți ajungând în același gameId.

Testele de joc verifică sincronizarea: o mutare făcută de jucătorul alb trebuie să apară pe tabla jucătorului negru după aproximativ o secundă, și invers. Se verifică și blocarea tablei: atunci când nu este rândul jucătorului local, tabla trebuie să fie inactivă. Urmează testarea ceasului (scăderea timpului doar pentru jucătorul la mutare), apoi testarea finalurilor de joc (remiză, cedare, flag) și a salvării în baza de date, inclusiv verificarea endpoint-urilor /game/history și /game/{gameId}/saved-moves. În final se testează modul „Allow illegal moves”.

Tabelul de mai jos prezintă un set de cazuri de testare. Coloana „Date de intrare” este inclusă în descrierea pașilor, iar coloana „Status” reflectă rezultatul așteptat; aceasta din urmă este o presupunere care trebuie confirmată la rularea efectivă a testelor.

Presupunere / Observație: Coloana „Status” și echivalentul „Rezultat obținut” din specificația cerută reflectă comportamentul așteptat dedus din cod, nu rezultate măsurate efectiv. La testarea reală, testerul ar trebui să completeze rezultatul obținut și să marcheze statusul ca PASS/FAIL în funcție de observații.

ID	Scop	Pași (și date de intrare)	Rezultat așteptat	Status
T01	Pornire MariaDB în Docker	Se rulează containerul MariaDB; se verifică portul și conexiunea	Baza de date rulează și acceptă conexiuni	Așteptat: PASS
T02	Pornire server Spring Boot	Se pornește ChessServerApplication	Serverul pornește fără erori, pe portul 8081	Așteptat: PASS
T03	Verificare /game/test	GET /game/test	Răspuns „Server merge!”	Așteptat: PASS
T04	Pornire primul client	Se pornește clientul, se apasă Start Game	Stare „Se caută adversar...”	Așteptat: PASS
T05	Matchmaking — al doilea client	Se pornește al doilea client, Start Game	Se formează partida (Game started)	Așteptat: PASS
T06	Același gameId	Se compară gameId la cei doi clienți	Ambii clienți au același	Așteptat: PASS

ID	Scop	Pași (și date de intrare)	Rezultat așteptat	Status
			gameId	
T07	Mutare cu alb	Albul mută o piesă (ex. pion e2-e4)	Mutarea apare local pe clientul alb	Așteptat: PASS
T08	Sincronizare la negru	Se observă tabla clientului negru	Mutarea albului apare în ~1s	Așteptat: PASS
T09	Mutare cu negru	Negrul mută o piesă	Mutarea apare local pe clientul negru	Așteptat: PASS
T10	Sincronizare la alb	Se observă tabla clientului alb	Mutarea negrului apare în ~1s	Așteptat: PASS
T11	Blocare tablă	Se încearcă mutarea când nu e tura proprie	Tabla este inactivă, mutarea nu se face	Așteptat: PASS
T12	Timer	Se urmărește ceasul în timpul jocului	Scade doar pentru jucătorul la mutare	Așteptat: PASS
T13	Remiză	Se apasă Remiză și se confirmă	Joc oprit, DRAW trimis, partidă salvată	Așteptat: PASS
T14	Cedare	Se apasă Cedează și se confirmă	Câștigă adversarul, rezultat salvat	Așteptat: PASS
T15	Flag mutare ilegală	În mod demo, mutare ilegală, apoi Flag	Câștigă adversarul, rezultat salvat	Așteptat: PASS
T16	Salvare în DB	Se finalizează o partidă	Apar înregistrări în tabelele de jocuri și mutări	Așteptat: PASS
T17	Verificare /game/history	GET /game/history	Lista jocurilor salvate, cu rezultate	Așteptat: PASS
T18	Verificare saved-moves	GET /game/{gameId}/saved-moves	Lista mutărilor ordonate după număr	Așteptat: PASS
T19	Allow illegal moves	Se activează opțiunea, se mută ilegal	Mutarea ilegală este permisă local	Așteptat: PASS
T20	Propagare final advers	Un client finalizează jocul	Celălalt client detectează finalul prin polling	Așteptat: PASS

3.14. Decizii de proiectare

Alegerile tehnologice și arhitecturale ale proiectului au fost făcute cu gândul la claritate și la valoarea didactică. JavaFX a fost ales pentru client deoarece este biblioteca grafică standard pentru aplicații desktop în Java, oferind controale bogate (tabele, butoane, etichete), un model de evenimente bine definit și mecanisme precum Timeline pentru animații și ceasuri. Spring Boot a fost ales pentru server pentru că reduce semnificativ efortul de configurare a unei aplicații web Java și oferă suport direct pentru controllere REST, injecție de dependențe și integrare cu JPA.

Comunicarea prin REST a fost preferată pentru simplitate și pentru că este un standard larg răspândit, ușor de testat manual (de exemplu cu un browser sau cu instrumente de tip client HTTP). Pentru sincronizare s-a ales polling în locul unui mecanism WebSocket complet, chiar dacă infrastructura WebSocket este parțial pregătită. Polling-ul este mai simplu de implementat și de înțeles, iar pentru un proiect didactic, întârzierea de aproximativ o secundă este acceptabilă. WebSocket-ul rămâne ca direcție de îmbunătățire.

MariaDB a fost aleasă ca bază de date relațională robustă și compatibilă cu ecosistemul JPA, iar rularea ei în Docker a fost preferată pentru izolare și reproductibilitate, eliminând nevoia unei instalări manuale. Separarea netă între client și server a fost o decizie de proiectare centrală: ea reflectă modelul real al aplicațiilor distribuite și permite ca fiecare componentă să fie dezvoltată și testată relativ independent.

Decizia de a face validarea regulilor de șah în principal pe client a fost luată pentru a simplifica serverul, care astfel se concentrează pe matchmaking, pe verificarea turei și pe persistență. Salvarea jocului doar la final (nu continuu) reduce traficul către baza de date și simplifică logica, cu prețul pierderii partidelor neterminate. În sfârșit, structurarea pe clase separate pentru UI, logică, comunicare, server și bază de date a fost o alegere conștientă, menită să mențină codul organizat și ușor de urmărit.

3.15. Limitări

Proiectul are o serie de limitări reale, asumate în mod transparent. Ele nu reduc valoarea didactică a aplicației, dar trebuie cunoscute pentru a înțelege corect ce face și ce nu face sistemul în forma actuală.

- Serverul nu validează complet legalitatea mutărilor de șah; el verifică doar existența jocului și tura jucătorului, ceea ce înseamnă că un client modificat ar putea trimite mutări ilegale respectând doar ordinea turelor.
- Sincronizarea se face prin polling, deci există o întârziere de aproximativ o secundă între mutarea unui jucător și apariția ei pe ecranul celuilalt.
- Jocurile active sunt ținute doar în memoria serverului (în `activeGames`); dacă serverul repornește, partidele în curs se pierd, fiind persistate doar cele finalizate.
- Nu există autentificare reală a utilizatorilor; identificarea se face printr-un `playerId` generat local, pe baza momentului curent.
- Remiza este, în implementarea curentă, aplicată imediat de jucătorul care o cere, fără o confirmare din partea adversarului, spre deosebire de regulile uzuale ale șahului.
- Nu există reconectare la joc după închiderea clientului; o instanță care se închide nu se poate reîntoarce în aceeași partidă.
- Nu există un lobby complex (selectarea adversarului, camere, opțiuni avansate); matchmaking-ul este o simplă coadă de tip primul-venit.
- Nu există un sistem de rating sau de clasament al jucătorilor.
- Opțiunea de culoare din pagina de setări nu determină culoarea efectivă în partidă; aceasta este atribuită de server în funcție de ordinea de intrare (`player1` = alb, `player2` = negru).
- WebSocket-ul este pregătit doar parțial (există apeluri de notificare pe server), dar nu este mecanismul principal de sincronizare; clientul folosește în continuare polling REST.

3.16. Posibile dezvoltări viitoare

Pornind de la limitările identificate, proiectul poate fi extins în mai multe direcții, multe dintre ele având deja un punct de plecare în codul actual.

- Implementarea unui mecanism WebSocket real pentru sincronizare instantanee, folosind infrastructura deja pregătită (`WebSocketConfig` și `WebSocketNotificationService`), pentru a elimina întârzierea polling-ului.

- Adăugarea unei autentificări reale a utilizatorilor, cu conturi și profiluri persistente, în locul identificatorilor generați local.
- Introducerea unui sistem de rating (de exemplu Elo) și a unui matchmaking în funcție de nivel.
- Realizarea unui lobby mai complex, cu posibilitatea de a crea și a alege partide.
- Mutarea validării complete a regulilor de șah și pe server, pentru a preveni manipularea clientului.
- Adăugarea posibilității de reconectare la joc după închiderea accidentală a clientului.
- Salvarea continuă a stării jocului (nu doar la final), pentru a permite recuperarea partidelor întrerupte.
- Implementarea unui mod spectator și a unui sistem de chat între jucători.
- Exportul partidelor în format PGN, standardul folosit de platformele de șah, pentru interoperabilitate.
- Integrarea cu un motor de șah (de exemplu pentru analiză post-partidă) și realizarea unei interfețe dedicate de analiză.

4. Concluzii

Proiectul a condus la realizarea unei aplicații de șah multiplayer complete din punct de vedere funcțional, construită după modelul client-server. Aplicația integrează un client JavaFX, un server Spring Boot și o bază de date MariaDB rulată în Docker, demonstrând astfel întregul lanț al unei aplicații distribuite, de la interfața grafică până la persistența datelor.

Pe parcursul implementării au fost atinse obiectivele propuse. A fost realizat un mecanism de matchmaking care pune în legătură doi jucători printr-o coadă de așteptare și o hartă a jocurilor active. A fost implementată sincronizarea mutărilor între cei doi clienți, bazată pe aplicarea locală a mutărilor și pe citirea periodică a stării jocului de la server, cu un contor (syncedMoveCount) care previne aplicarea dublă a aceleiași mutări. Partidele sunt salvate în baza de date la final, împreună cu lista de mutări. Fiecare jucător dispune de un ceas propriu, gestionat prin Timeline, care scade timpul doar pentru cel aflat la mutare. Au fost implementate finalurile speciale de joc — remiza, cedarea și flag-ul pentru mutări ilegale — precum și detectarea șah-matului și a patului, iar opțiunea „Allow illegal moves” oferă un mod de testare util pentru demonstrarea acestor mecanisme.

Față de o aplicație simplă de șah, jucată local pe un singur calculator, proiectul aduce o serie de elemente suplimentare semnificative: matchmaking între doi clienți, sincronizare client-server, salvare în MariaDB, istoric de mutări cu replay local, timer pentru fiecare jucător, finaluri speciale de joc și o interfață JavaFX personalizată, totul sprijinit de o bază de date containerizată. Aceste elemente transformă un exercițiu de logică a șahului într-un studiu practic de arhitectură software.

Din punct de vedere formativ, proiectul a oferit ocazia de a înțelege concret cum se construiește o aplicație distribuită: cum se proiectează endpoint-urile REST, cum se transmit și se sincronizează datele între instanțe, cum se separă responsabilitățile între straturi (UI, logică, comunicare, server, persistență) și cum se gestionează concurența pe firul de interfață în JavaFX. De asemenea, confruntarea cu limitările (precum validarea incompletă pe server sau întârzierea polling-ului) a evidențiat compromisurile inevitabile dintre simplitate și robustețe.

În concluzie, deși nu își propune să concureze cu platformele comerciale, proiectul își atinge scopul educațional: el demonstrează, într-un mod tangibil și verificabil, modul în care funcționează arhitecturile client-server. Tocmai prin faptul că fiecare componentă este vizibilă și poate fi modificată, aplicația devine un instrument valoros pentru înțelegerea principiilor care stau la baza majorității sistemelor software moderne.

Bibliografie

- [1] L. Whitwam, "Chess.com reaches 200 million members," TechCrunch, 24 apr. 2025. [Online]. Disponibil la: <https://techcrunch.com/2025/04/24/chess-com-reaches-200-million-members/> [Accesat: 27 mai 2026].
- [2] Chess.com, "Chess.com Reaches 250 Million Members," Chess.com News, feb. 2026. [Online]. Disponibil la: <https://www.chess.com/news/view/chesscom-reaches-250-million-members> [Accesat: 27 mai 2026].
- [3] Open Source For You, "Community-Powered Lichess Outplays Corporate Giants," 24 nov. 2025. [Online]. Disponibil la: <https://www.opensourceforu.com/2025/11/community-powered-lichess-outplays-corporate-giants/> [Accesat: 27 mai 2026].
- [4] Icon Era, "Chess Statistics 2026," citând Fortune Business Insights și FIDE. [Online]. Disponibil la: <https://icon-era.com/statistics/chess/> [Accesat: 27 mai 2026].
- [5] Oracle / OpenJFX, "JavaFX – Documentation," [Online]. Disponibil la: <https://openjfx.io/> [Accesat: 27 mai 2026].
- [6] VMware / Broadcom, "Spring Boot Reference Documentation," [Online]. Disponibil la: <https://spring.io/projects/spring-boot> [Accesat: 27 mai 2026].
- [7] MariaDB Foundation, "MariaDB Server Documentation," [Online]. Disponibil la: <https://mariadb.org/> [Accesat: 27 mai 2026].
- [8] Docker Inc., "Docker Documentation," [Online]. Disponibil la: <https://docs.docker.com/> [Accesat: 27 mai 2026].
- [9] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," teză de doctorat, Univ. California, Irvine, 2000 (definirea stilului REST).
- [10] Documentația și codul sursă ale proiectului (fișierele descproj.txt și classinputs.txt), sursă primară pentru descrierea și implementarea prezentate în această lucrare. [SURSĂ INTERNĂ — proiect].